

Approfondir — Pourquoi séparer le fichier et la règle

Après l'atelier. Le cours est `1-COURS.pdf` et l'exercice est `2-ATELIER.pdf` dans `projets/08-json/`.

Deux responsabilités différentes

Le JSON est pratique parce qu'il est simple à lire, copier et échanger. Cette simplicité vient d'une limite utile : il ne contient que des données. Une méthode JavaScript dans le fichier le rendrait invalide et l'attacherait à un seul langage.

La classe fait le travail inverse. Elle reçoit les données et garantit une manière commune de répondre ou d'agir. `estDispo()`, `emprunter(qui)` et `rendre()` restent cohérentes même si les matériels viennent plus tard d'une API ou d'une base de données.

L'aller-retour a une frontière

`JSON.parse(texte)` transforme du texte en valeurs JavaScript. Le constructeur transforme ensuite chaque valeur en objet `Materiel`. Dans l'autre sens, `JSON.stringify(...)` produit du texte à partir de l'état courant des objets.

Ce dernier texte n'est pas encore un fichier enregistré. Le navigateur protège votre disque : selon le projet, il faudra un téléchargement, un serveur ou une action manuelle. L'atelier choisit volontairement la copie manuelle pour rendre cette frontière visible.

```
fichier JSON → valeurs → objets Materiel → valeurs → texte JSON → fichier
```

Pourquoi la boucle ne doit pas connaître la règle

Imaginez que « disponible » signifie demain : aucun emprunteur **et** aucune maintenance prévue. Si la condition est recopiée dans l'affichage, le bouton et le compteur, trois endroits peuvent se contredire. Si elle vit dans `estDispo()`, tous les usages posent la même question au même objet.

C'est cette séparation qui prépare une application plus grande : le stockage change, l'interface change, mais le sens de « disponible » reste porté par le domaine.

Pour votre oral

Montrez une ligne du fichier et l'objet construit avec elle. Puis dites où l'état change pendant `emprunter(qui)` et pourquoi le JSON affiché doit encore être enregistré. Cette explication prouve davantage qu'une visite ligne par ligne du code.